Stacking & Blending Ensembles

➢ Introduction:

Ensemble methods combine multiple models to improve accuracy and robustness.
Stacking and **Blending** are advanced ensemble techniques.

➢ What is Stacking?

- **Stacking** = Training different base models (level-0 models), then using another model (meta-learner / level-1 model) to combine their predictions.
- Idea: Each model learns different patterns, and the meta-learner figures out how to best use them.

➢ The Problem with Stacking

- If you train meta-learner directly on base models' predictions from the same training data, it can **overfit** (because meta-learner "cheats" by seeing true labels too closely).

➢ Solutions

- To avoid leakage/overfitting → Use **out-of-fold predictions** (data not seen by base models) to train the meta-learner.

➢ Blending (Hold-Out Approach)

- Like stacking, but instead of folds:
    1. Split data into **train** and a **hold-out validation set**.
    2. Train base models on training set.
    3. Generate predictions on validation set → feed to meta-learner.
- Simpler but less data-efficient (since some data is kept aside).

➢ Stacking (K-Fold Approach)

- More robust than blending.
- Use **K-Fold cross-validation**:
    1. For each fold, train base models on k−1 folds and predict on the held-out fold
    2. Collect all out-of-fold predictions → use them to train meta-learner.
- Meta-learner never sees "cheating predictions."

➢ Multi-Layer Stacking

- Can extend stacking into **multiple levels**:

1. Level-0: Base models (e.g., Decision Tree, SVM, Logistic Regression).
2. Level-1: Meta-learner (e.g., Random Forest).
3. Level-2: Another meta-learner (e.g., Neural Net).

- Helps capture even more complex patterns.

➢ Key Takeaways

- **Stacking**: Uses multiple models + meta-learner → better predictions.
- **Blending**: A simpler variant using hold-out validation set.
- **K-Fold Stacking**: More robust, avoids overfitting.
- **Multi-layer stacking**: Multiple levels of learners.
- **SKLearn** makes it straightforward to implement.
